

EXERCISE

Ứng dụng Quản lý Tài liệu An toàn

Mô tả dự án:

Xây dựng ứng dụng web backend cho phép người dùng đăng nhập bằng tài khoản Google, sau đó tải lên các tệp tin (PDF, ảnh) và xem lại danh sách file đã upload. Toàn bộ kết nối đều được bảo mật qua HTTPS, và có chức năng redirect từ HTTP sang HTTPS.

Mục tiêu bài học:

- Củng cố kiến thức về cấu hình server HTTPS trong Node.js.
- Thực hành tạo khóa SSL tự ký và chứng chỉ công khai.
- Triển khai redirect từ HTTP sang HTTPS.
- Sử dụng Multer để xử lý upload file.
- Kết hợp OAuth với Google Passport để xác thực người dùng.
- Hiểu cách phối hợp nhiều kỹ thuật trong một ứng dụng thực tế.

Công nghệ sử dụng:

- **Node.js + Express:** Xây dựng server backend
- **Multer:** Upload file
- **Passport.js + Google OAuth 2.0:** Đăng nhập bằng tài khoản Google
- **HTTPS module:** Bảo mật giao tiếp giữa client và server
- **OpenSSL:** Tạo khóa private/public key
- **Nodemon:** Dev tool reload khi code thay đổi

Cấu trúc thư mục dự án:

```
secure-file-upload/
├── public/
│   ├── index.html
│   └── upload.html
├── uploads/           # Thư mục lưu file upload
├── ssl/               # Chứa key.pem & cert.pem
├── routes/
│   └── auth.js        # Route xác thực OAuth và upload
├── config/
│   └── passport.js    # Cấu hình Passport
├── .env               # Chứa biến môi trường
├── server.js          # Khởi động HTTPS và HTTP server chính
└── package.json
```



DANH SÁCH CÁC BƯỚC THỰC HIỆN

- **Bước 1:** Khởi tạo dự án và cài đặt dependencies
- **Bước 2:** Tạo khóa SSL với OpenSSL
- **Bước 3:** Cấu hình server HTTPS
- **Bước 4:** Cấu hình server HTTP để redirect sang HTTPS
- **Bước 5:** Cài đặt và cấu hình Multer để upload file
- **Bước 6:** Thiết lập OAuth với Google thông qua Passport.js
- **Bước 7:** Viết route xử lý upload và hiển thị file
- **Bước 8:** Tạo giao diện người dùng
- **Bước 9:** Chạy và kiểm thử ứng dụng



CHI TIẾT TỪNG BƯỚC



BƯỚC 1: KHỞI TẠO DỰ ÁN VÀ CÀI ĐẶT DEPENDENCIES

Mục tiêu: Chuẩn bị môi trường làm việc bằng cách tạo thư mục dự án và cài đặt các thư viện cần thiết.

Thao tác:

1. **Tạo thư mục dự án và di chuyển vào:**

```
mkdir secure-file-upload
```

```
cd secure-file-upload
```

2. **Khởi tạo dự án Node.js:**

```
npm init -y
```

3. **Cài đặt các dependencies chính:**

```
npm install express multer https http passport passport-google-oauth20 dotenv express-session
```

- **express:** Framework web cho Node.js.
- **multer:** Middleware để xử lý multipart/form-data (dùng cho upload file).
- **https:** Module tích hợp sẵn của Node.js để tạo server HTTPS.
- **http:** Module tích hợp sẵn của Node.js để tạo server HTTP.
- **passport:** Middleware xác thực linh hoạt cho Node.js.
- **passport-google-oauth20:** Chiến lược Passport cho Google OAuth 2.0.
- **dotenv:** Tải các biến môi trường từ file `.env` vào `process.env`.
- **express-session:** Middleware để quản lý phiên người dùng.

4. Cài đặt Nodemon (development dependency):

```
npm install --save-dev nodemon
```

nodemon: Tự động khởi động lại server khi có thay đổi trong mã nguồn.

Kiểm chứng: Mở file `package.json` và kiểm tra mục `dependencies` và `devDependencies`. Bạn sẽ thấy các gói đã cài đặt.

Cập nhật `package.json`: Thêm các script để dễ dàng chạy ứng dụng:

Mở file `package.json` và chỉnh sửa phần `"scripts"` như sau:

```
{
  "name": "secure-file-upload",
  "version": "1.0.0",
  "description": "",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "express-session": "^1.18.0",
    "http": "^0.0.1-security",
    "https": "^1.0.0",
    "multer": "^1.4.5-lts.1",
    "passport": "^0.7.0",
    "passport-google-oauth20": "^2.0.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.4"
  }
}
```

BƯỚC 2: TẠO KHÓA SSL VỚI OPENSSL

Mục tiêu: Tạo cặp khóa private key (`key.pem`) và public certificate (`cert.pem`) cần thiết để cấu hình server HTTPS. Chúng ta sẽ tạo chứng chỉ tự ký (self-signed certificate) cho mục đích phát triển.

Thao tác:

1. Tạo thư mục `ssl`:

```
mkdir ssl
```

2. Di chuyển vào thư mục `ssl`:

```
cd ssl
```

3. Sử dụng OpenSSL để tạo khóa và chứng chỉ:

```
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -days 365 -nodes
```

- `openssl req`: Lệnh tạo CSR (Certificate Signing Request) hoặc chứng chỉ tự ký.
- `-x509`: Chỉ định tạo chứng chỉ tự ký thay vì CSR.
- `-newkey rsa:4096`: Tạo một private key RSA mới với độ dài 4096 bit.
- `-keyout key.pem`: Lưu private key vào file `key.pem`.
- `-out cert.pem`: Lưu public certificate vào file `cert.pem`.
- `-days 365`: Đặt thời gian hiệu lực của chứng chỉ là 365 ngày.
- `-nodes`: Không mã hóa private key bằng passphrase. (Quan trọng cho việc tự động khởi động server).

Khi chạy lệnh, OpenSSL sẽ yêu cầu bạn nhập một số thông tin:

```
Country Name (2 letter code) [AU]:VN
State or Province Name (full name) [Some-State]:Can Tho
Locality Name (eg, city) []:Can Tho
Organization Name (eg, company) [Internet Widgits Pty Ltd]:MyCompany
Organizational Unit Name (eg, section) []:Development
Common Name (e.g. server FQDN or YOUR name) []:localhost
Email Address []:your.email@example.com
```

Bạn có thể điền thông tin bất kỳ hoặc để trống các trường nếu muốn, nhưng hãy đảm bảo Common Name là `localhost` để tránh cảnh báo trình duyệt khi truy cập trên máy cục bộ.

4. Quay lại thư mục gốc của dự án: `cd ..`

Kết quả: Trong thư mục `ssl/`, bạn sẽ thấy hai file:

- `key.pem`: Chứa private key của bạn.
- `cert.pem`: Chứa public certificate của bạn.

Kiểm chứng: Sử dụng lệnh `ls ssl/` để kiểm tra sự tồn tại của `key.pem` và `cert.pem`.

BƯỚC 3: CẤU HÌNH SERVER HTTPS

Mục tiêu: Tạo một server HTTPS sử dụng Express và các khóa SSL đã tạo.

Thao tác:

1. Tạo file `server.js` trong thư mục gốc của dự án:

```
// server.js
require('dotenv').config(); // Tải biến môi trường từ .env
const express = require('express');
const https = require('https');
const http = require('http'); // Thêm để cấu hình HTTP redirect
const fs = require('fs');
const path = require('path');
const session = require('express-session'); // Thêm session
const passport = require('passport'); // Thêm passport

const app = express();
```

```

// Đọc các file chứng chỉ SSL
const options = {
  key: fs.readFileSync(path.join(__dirname, 'ssl', 'key.pem')),
  cert: fs.readFileSync(path.join(__dirname, 'ssl', 'cert.pem'))
};

const PORT_HTTPS = 443; // Port mặc định cho HTTPS
const PORT_HTTP = 80; // Port mặc định cho HTTP

// Middleware để parse body của request
app.use(express.urlencoded({ extended: true }));
app.use(express.json());

// Cấu hình Express session
app.use(session({
  secret: 'a super secret key for session', // Thay đổi chuỗi này bằng một
chuỗi ngẫu nhiên mạnh
  resave: false,
  saveUninitialized: false,
  cookie: { secure: true } // Đảm bảo cookie chỉ gửi qua HTTPS
}));

// Khởi tạo Passport
app.use(passport.initialize());
app.use(passport.session());

// Import cấu hình Passport
require('./config/passport');

// Serve các file tĩnh từ thư mục public
app.use(express.static(path.join(__dirname, 'public')));
// Serve các file tĩnh từ thư mục uploads (để xem ảnh, PDF đã upload)
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

// Import route xác thực và upload
require('./routes/auth')(app);

// Khởi tạo server HTTPS
https.createServer(options, app).listen(PORT_HTTPS, () => {
  console.log(`HTTPS Server running on port ${PORT_HTTPS}`);
  console.log(`Access at: https://localhost`);
});

// Các routes khác và cấu hình HTTP redirect sẽ được thêm vào ở bước sau

```

Giải thích code:

- `require('dotenv').config();`: Tải các biến môi trường từ file `.env` (sẽ tạo ở bước sau).
- `fs.readFileSync()`: Đọc nội dung của file `key.pem` và `cert.pem`.
- `path.join(__dirname, 'ssl', 'key.pem')`: Xây dựng đường dẫn tuyệt đối đến file `key.pem`.
- `https.createServer(options, app)`: Tạo một server HTTPS với Express app và các chứng chỉ đã đọc.
- `app.use(express.static(path.join(__dirname, 'public')))`: Cho phép server phục vụ các file HTML, CSS, JS tĩnh từ thư mục `public`.
- `app.use('/uploads', express.static(path.join(__dirname, 'uploads')))`: Cho phép người dùng truy cập các file đã upload thông qua URL `/uploads/your-file.pdf`.

Kiểm chứng:

- Chạy `npm run dev`.
- Mở trình duyệt và truy cập `https://localhost`.
- Bạn có thể sẽ nhận được cảnh báo bảo mật vì đây là chứng chỉ tự ký. Bạn cần chấp nhận rủi ro hoặc thêm chứng chỉ vào danh sách tin cậy của hệ điều hành để tiếp tục.
- Nếu server chạy thành công, bạn sẽ thấy trang `index.html` được hiển thị và console log `HTTPS Server running on port 443`.

✅ BƯỚC 4: CẤU HÌNH SERVER HTTP ĐỂ REDIRECT SANG HTTPS

Mục tiêu: Đảm bảo rằng mọi yêu cầu HTTP đến ứng dụng sẽ tự động được chuyển hướng (redirect) sang HTTPS để tăng cường bảo mật.

Thao tác:

1. Thêm đoạn code sau vào cuối file `server.js` (sau khi khởi tạo server HTTPS):

```
// ... (code server.js ở trên) ...

// HTTP redirect to HTTPS
http.createServer((req, res) => {
  // res.writeHead(301, { "Location": "https://" + req.headers['host'] +
req.url });
  // Sửa lỗi khi host không bao gồm port
  const host = req.headers['host'].split(':')[0]; // Lấy hostname bỏ qua port
nếu có
  res.writeHead(301, { "Location": "https://" + host + req.url });
  res.end();
}).listen(PORT_HTTP, () => {
  console.log(`HTTP redirect server running on port ${PORT_HTTP}`);
  console.log(`Access HTTP at: http://localhost`);
});
```

Giải thích code:

- `http.createServer(...)`: Tạo một server HTTP.
- `res.writeHead(301, { "Location": "https://" + req.headers['host'] + req.url })`: Gửi mã trạng thái 301 Moved Permanently và header Location để trình duyệt tự động chuyển hướng sang URL HTTPS tương ứng.
- `req.headers['host']`: Lấy hostname và port (nếu có) từ request HTTP ban đầu.
- `req.url`: Lấy đường dẫn URL từ request HTTP ban đầu.
- `host.split(':')[0]`: Xử lý trường hợp `req.headers['host']` có thể bao gồm cả port (ví dụ: `localhost:80`), chúng ta chỉ cần phần hostname để tránh lỗi `localhost:80:443`.

Kiểm chứng:

- Chạy `npm run dev` nếu chưa chạy.
- Mở trình duyệt và truy cập `http://localhost`.
- Kiểm tra xem trình duyệt có tự động chuyển hướng sang `https://localhost` hay không. Nếu có, cấu hình đã thành công.

✅ BƯỚC 5: CÀI ĐẶT VÀ CẤU HÌNH MULTER

Mục tiêu: Chuẩn bị thư mục để lưu trữ các file đã upload và cấu hình Multer để xử lý việc tải lên file.

Thao tác:

1. **Tạo thư mục `uploads`** trong thư mục gốc của dự án:

```
mkdir uploads
```

1. Thư mục này sẽ là nơi Multer lưu trữ các file được người dùng tải lên.
2. **Multer sẽ được cấu hình trong `routes/auth.js`** (sẽ tạo ở Bước 7).

Kiểm chứng: Sử dụng lệnh `ls` trong thư mục gốc của dự án, bạn sẽ thấy thư mục `uploads` đã được tạo.

BƯỚC 6: THIẾT LẬP OAUTH VỚI GOOGLE THÔNG QUA PASSPORT.JS

Mục tiêu: Cấu hình Passport.js để tích hợp xác thực Google OAuth 2.0, cho phép người dùng đăng nhập bằng tài khoản Google của họ.

Thao tác:

1. **Tạo file `.env` ở thư mục gốc của dự án:** File này sẽ chứa các biến môi trường nhạy cảm như Client ID và Client Secret của Google.

```
GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret
```

• Lưu ý quan trọng:

- Bạn cần phải tạo một dự án trên Google Cloud Console để lấy `GOOGLE_CLIENT_ID` và `GOOGLE_CLIENT_SECRET`.
- Truy cập [Google Cloud Console](https://console.cloud.google.com/).
- Tạo một dự án mới.
- Trong "APIs & Services" -> "Credentials", tạo một "OAuth client ID".
- Chọn "Web application".
- Thêm `https://localhost/auth/google/callback` vào "Authorized redirect URIs".
- Sau khi tạo, bạn sẽ nhận được Client ID và Client Secret. Thay thế `your_google_client_id` và `your_google_client_secret` trong file `.env` bằng các giá trị này.

2. **Tạo thư mục `config`:**

```
mkdir config
```

3. Tạo file `config/passport.js`: File này chứa logic cấu hình Passport cho Google Strategy.

```
// config/passport.js
const passport = require('passport');
const GoogleStrategy = require('passport-google-oauth20').Strategy;

// Cấu hình Google Strategy
passport.use(new GoogleStrategy({
  clientID: process.env.GOOGLE_CLIENT_ID,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET,
  callbackURL: "/auth/google/callback" // URL callback sau khi xác thực thành công
},
  function(token, refreshToken, profile, done) {
    // Hàm callback khi người dùng đã xác thực thành công với Google
    // `profile` chứa thông tin người dùng từ Google (id, name, email, photos...)
    // `done` là hàm callback để thông báo cho Passport đã hoàn thành xác thực
    // Trong ứng dụng thực tế, bạn sẽ lưu `profile` vào cơ sở dữ liệu
    // Ở đây, chúng ta chỉ đơn giản trả về profile
    return done(null, profile);
  }
));

// Serialize user: Lưu thông tin người dùng vào session
// Ở đây chúng ta lưu toàn bộ đối tượng user (profile từ Google)
passport.serializeUser((user, done) => {
  done(null, user);
});

// Deserialize user: Lấy thông tin người dùng từ session
// Ở đây chúng ta lấy lại đối tượng user từ session
passport.deserializeUser((obj, done) => {
  done(null, obj);
});
```

Giải thích code:

- `GoogleStrategy`: Định nghĩa cách Passport sẽ xác thực người dùng thông qua Google OAuth 2.0.
- `clientID` và `clientSecret`: Lấy từ biến môi trường (được tải bởi `dotenv`).
- `callbackURL`: Đường dẫn mà Google sẽ redirect người dùng trở lại sau khi họ đã xác thực.
- `serializeUser`: Quyết định dữ liệu nào của người dùng sẽ được lưu vào session.
- `deserializeUser`: Quyết định cách lấy lại thông tin người dùng từ dữ liệu đã lưu trong session.

Kiểm chứng: Kiểm tra xem bạn đã thêm `require('dotenv').config();`, `express-session` và `passport` vào `server.js` ở Bước 3 chưa. Bạn có thể tạm thời chạy `console.log(process.env.GOOGLE_CLIENT_ID);` trong `server.js` để kiểm tra xem các biến môi trường đã được tải đúng cách chưa (sau đó xóa dòng này đi).

BƯỚC 7: VIẾT ROUTE XỬ LÝ UPLOAD FILE VÀ XÁC THỰC

Mục tiêu: Tạo các route để xử lý quá trình đăng nhập Google OAuth và route để xử lý việc upload file bằng Multer.

Thao tác:

1. Tạo thư mục routes:

```
mkdir routes
```

2. Tạo file routes/auth.js: File này sẽ chứa các route liên quan đến xác thực và upload.

```
// routes/auth.js
const express = require('express');
const multer = require('multer');
const path = require('path');
const passport = require('passport'); // Đảm bảo passport được import

module.exports = (app) => {

  // Cấu hình Multer để lưu trữ file
  const storage = multer.diskStorage({
    destination: './uploads/', // Thư mục đích để lưu file
    filename: (req, file, cb) => {
      // Tạo tên file duy nhất: timestamp-originalfilename
      cb(null, Date.now() + '-' + file.originalname);
    }
  });

  // Khởi tạo Multer upload middleware
  const upload = multer({
    storage: storage,
    limits: { fileSize: 1024 * 1024 * 10 }, // Giới hạn kích thước file 10MB
    fileFilter: (req, file, cb) => {
      // Kiểm tra loại file
      const filetypes = /jpeg|jpg|png|pdf/;
      // Kiểm tra phần mở rộng file
      const extname =
        filetypes.test(path.extname(file.originalname).toLowerCase());
      // Kiểm tra kiểu MIME
      const mimetype = filetypes.test(file.mimetype);

      if (mimetype && extname) {
        return cb(null, true); // Chấp nhận file
      } else {
        cb('Error: Only JPEG, JPG, PNG, and PDF files are allowed!'); // Từ chối file
      }
    }
  });

  // Route cho Google OAuth login
  app.get('/auth/google',
    passport.authenticate('google', { scope: ['profile', 'email'] }));

  // Route callback sau khi Google xác thực
  app.get('/auth/google/callback',
    passport.authenticate('google', { failureRedirect: '/' }), // Nếu thất bại,
    // redirect về trang chủ
    (req, res) => {
      // Nếu xác thực thành công, redirect đến trang upload
      res.redirect('/upload');
    });

  // Route logout
  app.get('/logout', (req, res, next) => {
    req.logout((err) => { // Dùng callback cho req.logout()
      if (err) { return next(err); }
      res.redirect('/');
    });
  });

  // Middleware để kiểm tra xem người dùng đã đăng nhập chưa
```

```

function isAuthenticated(req, res, next) {
  if (req.isAuthenticated()) { // req.isAuthenticated() được cung cấp bởi Passport
    return next();
  }
  res.redirect('/'); // Nếu chưa đăng nhập, redirect về trang chủ
}

// Route hiển thị trang upload (chỉ dành cho người dùng đã đăng nhập)
app.get('/upload', isAuthenticated, (req, res) => {
  res.sendFile(path.join(__dirname, '..', 'public', 'upload.html'));
});

// Route xử lý upload file (chỉ dành cho người dùng đã đăng nhập)
app.post('/upload', isAuthenticated, upload.single('file'), (req, res) => {
  if (!req.file) {
    return res.status(400).send('No file uploaded.');
```

```

  }
  // Sau khi upload thành công, có thể hiển thị danh sách file
  // hoặc thông báo thành công
  res.send(`File "<span class="math-inline">\{req.file.filename\}" uploaded successfully\! <a href="/uploads/</span>\{req.file.filename}" target="_blank">View File</a><br><a href="/upload">Upload another file</a><br><a href="/logout">Logout</a>`);
});
```

```

// Route để hiển thị danh sách các file đã upload (tùy chọn)
app.get('/files', isAuthenticated, (req, res) => {
  fs.readdir(path.join(__dirname, '..', 'uploads'), (err, files) => {
    if (err) {
      return res.status(500).send('Unable to scan files!');
    }
    let fileListHtml = '<h2>Your Uploaded Files:</h2><ul>';
    if (files.length === 0) {
      fileListHtml += '<li>No files uploaded yet.</li>';
    } else {
      files.forEach(file => {
        fileListHtml += '<li><a href="/uploads/<span class="math-inline">\{file\}" target="\_blank"></span>\{file}</a></li>';
      });
    }
    fileListHtml += '</ul><p><a href="/upload">Upload new file</a> | <a href="/logout">Logout</a></p>`;
    res.send(fileListHtml);
  });
});
};
```

Giải thích code:

- `multer.diskStorage`: Định nghĩa nơi lưu trữ file và cách đặt tên file.
- `upload.single('file')`: Middleware của Multer, xử lý một file duy nhất với tên trường file từ form.
- `fileFilter`: Hàm kiểm tra loại file cho phép tải lên.
- `passport.authenticate('google', ...)`: Middleware của Passport để bắt đầu quá trình xác thực Google.
- `isAuthenticated`: Một middleware tùy chỉnh để kiểm tra trạng thái đăng nhập của người dùng.
- `/logout`: Route để người dùng đăng xuất khỏi ứng dụng và xóa session.
- `/files`: (Tùy chọn) Một route để liệt kê các file đã được upload.

Kiểm chứng: Đảm bảo bạn đã thêm `require('./routes/auth')(app);` vào `server.js` ở Bước 3.

✓ BƯỚC 8: TẠO GIAO DIỆN NGƯỜI DÙNG

Mục tiêu: Tạo các file HTML đơn giản để người dùng có thể tương tác với ứng dụng (trang đăng nhập và trang upload).

Thao tác:

1. Tạo thư mục `public`:

```
mkdir public
```

2. Tạo file `public/index.html`: Trang chào mừng và nút đăng nhập.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Secure File Upload - Login</title>
  <style>
    body { font-family: Arial, sans-serif; display: flex; justify-content:
center; align-items: center; height: 100vh; margin: 0; background-color: #f4f4f4; }
    .container { background-color: #fff; padding: 30px; border-radius: 8px;
box-shadow: 0 2px 10px rgba(0,0,0,0.1); text-align: center; }
    h2 { color: #333; margin-bottom: 20px; }
    a.google-login-btn {
      display: inline-block;
      background-color: #4285F4;
      color: white;
      padding: 10px 20px;
      border-radius: 5px;
      text-decoration: none;
      font-weight: bold;
      transition: background-color 0.3s ease;
    }
    a.google-login-btn:hover {
      background-color: #357ae8;
    }
  </style>
</head>
<body>
  <div class="container">
    <h2>Welcome to Secure File Upload</h2>
    <p>Please log in with your Google account to proceed.</p>
    <a href="/auth/google" class="google-login-btn">Login with Google</a>
  </div>
</body>
</html>
```

3. Tạo file `public/upload.html`: Trang upload file sau khi đăng nhập thành công.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Upload Your File</title>
  <style>
    body { font-family: Arial, sans-serif; display: flex; flex-direction:
column; justify-content: center; align-items: center; height: 100vh; margin: 0;
background-color: #f4f4f4; }
    .container { background-color: #fff; padding: 30px; border-radius: 8px;
box-shadow: 0 2px 10px rgba(0,0,0,0.1); text-align: center; }
```

```

h2 { color: #333; margin-bottom: 20px; }
form { margin-bottom: 20px; }
input[type="file"] { margin-bottom: 15px; }
button[type="submit"] {
  background-color: #28a745;
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  font-size: 16px;
  transition: background-color 0.3s ease;
}
button[type="submit"]:hover {
  background-color: #218838;
}
.links a {
  margin: 0 10px;
  color: #007bff;
  text-decoration: none;
}
.links a:hover {
  text-decoration: underline;
}
</style>
</head>
<body>
  <div class="container">
    <h2>Upload your file</h2>
    <form action="/upload" method="POST" enctype="multipart/form-data">
      <input type="file" name="file" required/>
      <button type="submit">Upload</button>
    </form>
    <div class="links">
      <p><a href="/files">View Uploaded Files</a> | <a
href="/logout">Logout</a></p>
    </div>
  </div>
</body>
</html>

```

Kiểm chứng: Kiểm tra sự tồn tại của các file `index.html` và `upload.html` trong thư mục `public/`.

✅ BƯỚC 9: CHẠY VÀ KIỂM THỬ ỨNG DỤNG

Mục tiêu: Khởi động ứng dụng Node.js và kiểm tra từng chức năng đã triển khai.

Thao tác:

1. **Chạy ứng dụng:** Mở terminal trong thư mục gốc của dự án và chạy:

```
npm run dev
```

- Bạn sẽ thấy các thông báo trong console cho biết cả server HTTPS và HTTP redirect đang chạy.

- **Kiểm tra từng chức năng:**

- **HTTP Redirect sang HTTPS:**

- Mở trình duyệt và truy cập: `http://localhost`
- **Kiểm chứng:** Trình duyệt của bạn sẽ tự động chuyển hướng đến `https://localhost`. Nếu có cảnh báo về chứng chỉ không đáng tin cậy (do là chứng chỉ tự ký), hãy chấp nhận để tiếp tục.
- **Google OAuth Login:**
 - Sau khi chuyển hướng đến `https://localhost`, bạn sẽ thấy trang `index.html` với nút "Login with Google".
 - Nhấp vào "Login with Google".
 - **Kiểm chứng:** Bạn sẽ được chuyển hướng đến trang đăng nhập của Google. Đăng nhập bằng tài khoản Google của bạn. Sau khi đăng nhập thành công, bạn sẽ được chuyển hướng trở lại `/auth/google/callback` và sau đó đến trang `/upload`.
 - Nếu bạn đã đăng nhập thành công, URL sẽ là `https://localhost/upload` và bạn sẽ thấy trang "Upload your file".
- **Upload File bằng Multer:**
 - Trên trang `https://localhost/upload`, chọn một file (ảnh JPG/PNG hoặc PDF) từ máy tính của bạn.
 - Nhấp vào nút "Upload".
 - **Kiểm chứng:**
 - Bạn sẽ nhận được thông báo "File uploaded successfully!" cùng với một liên kết để xem file.
 - Kiểm tra thư mục `uploads/` trong dự án của bạn. Bạn sẽ thấy file đã tải lên với tên được tạo bởi Multer (`timestamp-originalfilename`).
 - Nhấp vào liên kết "View File" để đảm bảo file có thể xem được thông qua URL `/uploads/your-file-name.pdf`.
- **Xem danh sách file đã upload:**
 - Nhấp vào liên kết "View Uploaded Files" trên trang `upload`.
 - **Kiểm chứng:** Bạn sẽ được đưa đến trang `/files` hiển thị danh sách các file đã upload.
- **Logout:**
 - Nhấp vào liên kết "Logout" trên trang `upload` hoặc trang file list.
 - **Kiểm chứng:** Bạn sẽ được chuyển hướng trở lại trang `https://localhost` (`index.html`), và session của bạn sẽ bị xóa. Nếu bạn cố gắng truy cập `/upload` hoặc `/files` ngay sau khi đăng xuất, bạn sẽ bị chuyển hướng trở lại trang đăng nhập.



Tổng hợp Hướng Dẫn Thực Thi và Kiểm Chứng

Trước khi bắt đầu, hãy đảm bảo bạn đã hoàn thành tất cả các bước cài đặt và cấu hình theo hướng dẫn trước, và bạn đang ở thư mục gốc của dự án `secure-file-upload`.

1. Khởi động ứng dụng

Đầu tiên, chúng ta cần chạy ứng dụng Node.js của bạn.

Thao tác: Mở cửa sổ Terminal hoặc Command Prompt, điều hướng đến thư mục `secure-file-upload` và chạy lệnh:

```
npm run dev
```

Kết quả mong đợi: Bạn sẽ thấy output tương tự như sau trong terminal:

```
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
HTTPS Server running on port 443
Access at: https://localhost
HTTP redirect server running on port 80
Access HTTP at: http://localhost
```

Điều này cho thấy cả hai server HTTP (chuyển hướng) và HTTPS đều đang hoạt động.

2. Kiểm chứng kỹ thuật: HTTPS

Mục tiêu: Đảm bảo server HTTPS hoạt động và có thể truy cập được.

Thao tác:

1. Mở trình duyệt web của bạn (Chrome, Firefox, Edge...).
2. Nhập địa chỉ: **https://localhost**

Kết quả mong đợi:

- Bạn có thể sẽ nhận được một cảnh báo bảo mật từ trình duyệt (ví dụ: "Your connection is not private" hoặc "Potential Security Risk Ahead"). Đây là điều **hoàn toàn bình thường** vì bạn đang sử dụng một **chứng chỉ SSL tự ký** (self-signed certificate) mà trình duyệt không tin tưởng mặc định.
- Bạn cần **chấp nhận rủi ro và tiếp tục** (ví dụ: nhấp vào "Advanced" hoặc "More Information", sau đó chọn "Proceed to localhost" hoặc "Accept the risk and Continue").
- Sau khi chấp nhận, bạn sẽ thấy trang `index.html` của ứng dụng được hiển thị với nội dung "Welcome to Secure File Upload".

Đánh giá: Nếu bạn thấy trang web được tải qua HTTPS sau khi vượt qua cảnh báo chứng chỉ, tức là server HTTPS đã được cấu hình và hoạt động thành công.

3. Kiểm chứng kỹ thuật: Redirect HTTP → HTTPS

Mục tiêu: Xác minh rằng mọi yêu cầu HTTP đến ứng dụng sẽ tự động được chuyển hướng sang HTTPS.

Thao tác:

1. Mở một tab trình duyệt mới.
2. Nhập địa chỉ: **http://localhost** (lưu ý: là `http` chứ không phải `https`).

Kết quả mong đợi:

- Ngay lập tức, trình duyệt của bạn sẽ tự động chuyển hướng sang **https://localhost**.
- Thanh địa chỉ sẽ hiển thị `https://localhost` (có thể vẫn kèm theo cảnh báo chứng chỉ nếu bạn chưa thêm ngoại lệ).

Đánh giá: Nếu bạn thấy địa chỉ URL tự động thay đổi từ `http` sang `https`, điều đó có nghĩa là server HTTP redirect đã hoạt động chính xác.

4. Kiểm chứng kỹ thuật: OAuth Google

Mục tiêu: Đảm bảo tính năng đăng nhập bằng tài khoản Google hoạt động như mong đợi.

Thao tác:

1. Từ trang `https://localhost` (trang `index.html`), nhấp vào nút **"Login with Google"**.
2. Bạn sẽ được chuyển hướng đến trang đăng nhập của Google. Chọn một tài khoản Google để đăng nhập (hoặc đăng nhập nếu chưa).
3. Sau khi đăng nhập thành công, bạn sẽ được chuyển hướng trở lại ứng dụng của mình.

Kết quả mong đợi:

- Bạn sẽ được chuyển hướng đến địa chỉ `https://localhost/upload`.
- Trang "Upload your file" sẽ hiển thị. Điều này chứng tỏ Passport.js và Google OAuth đã xác thực bạn thành công và chuyển hướng đến trang được bảo vệ.

Đánh giá: Nếu bạn có thể đăng nhập bằng Google và được chuyển đến trang upload, tính năng OAuth đã hoạt động.

5. Kiểm chứng kỹ thuật: Upload File (Multer)

Mục tiêu: Xác minh khả năng upload file bằng Multer và kiểm tra file đã được lưu trữ đúng cách.

Thao tác:

1. Trên trang `https://localhost/upload`, bạn sẽ thấy một trường `Choose File` và nút `Upload`.
2. Nhấp vào `Choose File` và chọn một tệp tin hợp lệ (ví dụ: một ảnh `.jpg`, `.png` hoặc một tệp `.pdf`) từ máy tính của bạn.
3. Nhấp vào nút **"Upload"**.

Kết quả mong đợi:

- Trình duyệt sẽ hiển thị thông báo thành công: `File "ten-file-cua-ban" uploaded successfully! View File.`
- Kiểm tra thư mục `uploads/` trong thư mục gốc của dự án Node.js của bạn. Bạn sẽ thấy tệp tin vừa tải lên đã được lưu trữ ở đó, với tên mới (ví dụ: `1718612345678-my-image.png`).
- Nhấp vào liên kết **"View File"** trên trình duyệt. Thao tác này sẽ mở tệp tin của bạn trong một tab mới, chứng tỏ server đang phục vụ các file từ thư mục `uploads` một cách chính xác.

Đánh giá: Nếu file của bạn được lưu trong thư mục `uploads/` và có thể xem được qua trình duyệt, Multer và cơ chế phục vụ file tĩnh đã hoạt động thành công.

6. Kiểm chứng chức năng Logout

Mục tiêu: Đảm bảo người dùng có thể đăng xuất và session của họ bị hủy.

Thao tác:

1. Từ trang upload (<https://localhost/upload>) hoặc trang xem file (<https://localhost/files>), nhấp vào liên kết "**Logout**".

Kết quả mong đợi:

- Bạn sẽ được chuyển hướng trở lại trang đăng nhập (<https://localhost>).
- Nếu bạn thử truy cập lại <https://localhost/upload> hoặc <https://localhost/files> sau khi đăng xuất, bạn sẽ bị chuyển hướng về trang `index.html` (đăng nhập), cho thấy bạn đã không còn được xác thực.

Đánh giá: Nếu bạn bị đưa về trang đăng nhập sau khi nhấp "Logout" và không thể truy cập lại các trang được bảo vệ, chức năng đăng xuất đã hoạt động tốt.

